UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

LABORATORIO DE COMPUTACIÓN I

Unidad Temática N°4:
Consultas Continuación

Material Teórico
1° Año – 1° Cuatrimestre

CC BY ND NC

Índice

Funciones de agregado y consultas sumarias 3

Consultas agrupadas 6

Introducción a los procedimientos almacenados 10

Creación y ejecución de procedimientos almacenados 11

Modificación de procedimientos almacenados 12

Eliminación de procedimientos almacenados 14

Procedimientos almacenados con parámetros de entrada 15

BIBLIOGRAFÍA 25

.

Funciones de agregado y consultas sumarias

Las funciones agregadas o sumarias, también conocidas como funciones de agregación, son funciones en SQL que operan en conjuntos de datos y devuelven un único valor que resume o sintetiza la información de esos datos. Estas funciones realizan cálculos estadísticos o matemáticos en columnas o grupos de filas de una tabla y proporcionan resultados basados en esos cálculos.

Las funciones agregadas permiten realizar operaciones como sumar valores, calcular promedios, contar filas, encontrar el valor máximo o mínimo, entre otros. Estas funciones son muy útiles para obtener información resumida de grandes conjuntos de datos y realizar análisis de datos.

Algunas de las funciones agregadas comunes son:

- **SUM**: Calcula la suma de los valores en una columna numérica.
- **AVG**: Calcula el promedio de los valores en una columna numérica.
- **COUNT**: Cuenta el número de filas o valores no nulos en una columna.
- **MAX**: Devuelve el valor máximo de una columna.
- **MIN**: Devuelve el valor mínimo de una columna.

Estas funciones se utilizan en combinación con cláusulas como GROUP BY y HAVING para realizar consultas sumarias más complejas y obtener resultados agregados basados en ciertos criterios o grupos.

Vemos un ejemplo de la base de datos Librería: Se desea obtener información específica sobre la factura número 236 en una base de dato. Se requiere conocer lo siguiente:

- El **importe total** de la factura número 236.
- La **cantidad de artículos vendidos** en esa factura.
- La **cantidad de ventas realizadas** (número de registros) en la tabla "detalle_facturas" para la factura número 236.
- El **precio máximo vendido** en la factura número 236.
- El **precio mínimo vendido** en la factura número 236.

Para obtener esta información, se puede utilizar la siguiente consulta SQL:

```
SELECT SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL',
       SUM(DF.CANTIDAD) 'CANTIDAD ARTICULOS',
       COUNT(*) 'CANTIDAD VENTAS',
       MAX(DF.PRE_UNITARIO) 'PRECIO MAXIMO',
       MIN(DF.PRE_UNITARIO) 'PRECIO MINIMO'
  FROM DETALLE_FACTURAS DF
 WHERE DF.NRO_FACTURA = 236;
```

IMPORTE TOTAL	CANTIDAD ARTICULOS	CANTIDAD VENTAS	PRECIO MAXIMO	PRECIO MINIMO
1351.00	52	2	31.00	22.30

Imagen 1: Elaboración propia.

A continuación, se explica cada parte de la consulta:

- **SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL'**: Calcula el importe total de la factura multiplicando el precio unitario (PRE_UNITARIO) por la cantidad (CANTIDAD) de cada artículo vendido en la factura. La función SUM() suma todos los importes totales y lo nombra como 'IMPORTE TOTAL' en el resultado de la consulta.
- **SUM(DF.CANTIDAD) 'CANTIDAD ARTICULOS'**: Calcula la cantidad total de artículos vendidos en la factura sumando la columna CANTIDAD de la tabla DETALLE_FACTURAS. El resultado se nombra como 'CANTIDAD ARTICULOS'.
- **COUNT(*) 'CANTIDAD_VENTAS'**: Cuenta el número total de registros en la tabla DETALLE_FACTURAS relacionados con la factura número 236, lo cual representa la cantidad de ventas realizadas. El resultado se nombra como 'CANTIDAD_VENTAS'.
- **MAX(DF.PRE_UNITARIO) 'PRECIO MAXIMO'**: Determina el precio máximo (PRE_UNITARIO) de todos los registros relacionados con la factura número 236 en la tabla DETALLE_FACTURAS. El resultado se nombra como 'PRECIO MAXIMO'.
- **MIN(DF.PRE_UNITARIO) 'PRECIO MINIMO'**: Determina el precio mínimo (PRE_UNITARIO) de todos los registros relacionados con la factura número 236 en la tabla DETALLE_FACTURAS. El resultado se nombra como 'PRECIO MINIMO'.

En SQL, los agregados **AVG (promedio)**, **COUNT (conteo)** y **SUM (suma)** se pueden aplicar a una columna específica para realizar cálculos sobre sus valores. Cuando se agrega la palabra clave **DISTINCT** antes de la columna en estos agregados, se realiza el cálculo solo sobre los valores únicos de esa columna, es decir, se eliminan los duplicados antes de realizar el cálculo.

Por lo tanto, los agregados **AVG (DISTINCT columna)**, **COUNT (DISTINCT columna)** y **SUM (DISTINCT columna)** se denominan agregados ***distinct***, ya que se aplican a los valores distintos (únicos) de una columna en lugar de considerar todos los valores, incluidos los duplicados.

A continuación, se presenta una breve descripción de cada uno de estos agregados:

- **AVG (DISTINCT columna)**: Calcula el promedio de los valores únicos de la columna especificada. Los valores duplicados se eliminan antes de realizar el cálculo del promedio.
- **COUNT (DISTINCT columna):** Cuenta la cantidad de valores únicos en la columna especificada. Los valores duplicados se eliminan antes de realizar el conteo.
- **SUM (DISTINCT columna):** Calcula la suma de los valores únicos en la columna especificada. Los valores duplicados se eliminan antes de realizar la suma.

Estos agregados **distinct** son útiles cuando se desea realizar cálculos sobre los valores únicos de una columna en lugar de considerar todos los valores, lo que puede ser especialmente relevante cuando se trabaja con conjuntos de datos que contienen duplicados.

Consultas agrupadas

Cláusula GROUP BY

Las consultas sumarias se asemejan a los totales finales de un informe. Si se desea resumir u obtener resúmenes de columnas de la consulta a un nivel de subtotal, es decir, por cada cambio en una columna en particular, se utiliza la cláusula **GROUP BY** en la sentencia **SELECT**.

En otras palabras, se agrupan los registros según los valores de una o más columnas y se obtienen totales o resúmenes para cada grupo. Una consulta que incluye la cláusula **GROUP BY** se denomina consulta agrupada, ya que agrupa los datos de las tablas fuente y produce una única fila sumaria por cada grupo de filas.

Las columnas especificadas en la cláusula GROUP BY se conocen como columnas de agrupación de la consulta, ya que son las que determinan cómo se dividen las filas en grupos.

La sintaxis básica de la sentencia **GROUP BY** es la siguiente:

```
SELECT columna1, columna2, ..., columnaN
  FROM tabla
 GROUP BY columna1, columna2, ..., columnaN;
```

En esta sintaxis, "columna1, columna2, ..., columnaN" representa las columnas por las cuales deseas agrupar los registros. Puedes incluir tantas columnas como desees. *Es importante destacar que todas las columnas utilizadas en la cláusula **SELECT** deben estar presentes en la cláusula **GROUP BY**. Sin embargo, las columnas incluidas en la cláusula **GROUP BY** no necesariamente deben aparecer en la cláusula **SELECT**.*

Si se incluyen funciones agregadas en la lista **SELECT, GROUP BY** calcula un valor resumen para cada grupo. Estos se conocen como agregados vectoriales.

Veamos un ejemplo sencillo para comprenderlo mejor. Supongamos que trabajamos con una base de datos llamada "librería" y *queremos determinar la cantidad de clientes por barrio*.

En primer lugar, necesitamos realizar una operación de unión (join) entre las tablas "barrios" y "clientes" para obtener únicamente el nombre del barrio.

La consulta sería la siguiente:

```
SELECT B.BARRIO
  FROM CLIENTES C, BARRIOS B
 WHERE C.COD_BARRIO = B.COD_BARRIO
```

Esta consulta traerá únicamente los clientes que tengan un barrio asociado.

	BARRIO
1	CENTRO
2	PARQUE HORIZONTE
3	PANAMERICANO
4	ALTO ALBERDI
5	ALTO ALBERDI
6	ALTO ALBERDI
7	CENTRO
8	OBSERVATORIO
9	CENTRO
10	CENTRO

Imagen 2: Elaboración propia.

Sin embargo, para saber cuántos clientes hay por cada barrio, utilizaremos la función **COUNT(*)** junto con la cláusula **GROUP BY**. La consulta quedaría así:

```
SELECT B.BARRIO, COUNT(*) 'Cant. Clientes x Barrio'
  FROM CLIENTES C, BARRIOS B
 WHERE C.COD_BARRIO = B.COD_BARRIO
 GROUP BY B.BARRIO;
```

En esta consulta, estamos seleccionando el nombre del barrio (B.BARRIO) y contando la cantidad de registros por cada barrio utilizando COUNT(*). Además, utilizamos la cláusula GROUP BY para agrupar los resultados por barrio.

	BARRIO	Cant. Clientes x Barrio
1	ALTO ALBERDI	4
2	CENTRO	5
3	MAIPÚ	1
4	NUEVA CÓRDOBA	2
5	OBSERVATORIO	1
6	PANAMERICANO	1
7	PARQUE HORIZONTE	1

Imagen 3: Elaboración propia.

La pregunta que nos haríamos es por qué no hay datos de cliente en la consulta, o si los hay de manera indirecta. La razón por la cual no se muestran los datos de los clientes en la consulta original es que no se incluyó la proyección de los datos de la tabla "clientes". Para contar la cantidad de clientes por barrio, es necesario agrupar los datos de manera adecuada.

Si se intentara agrupar directamente por los campos de nombres y apellidos de los clientes, se contarían por separado a pesar de ser la misma persona, lo que generaría resultados incorrectos y contradeciría el propósito del "GROUP BY".

Múltiples columnas de agrupación.

En la cláusula **GROUP BY** de una consulta SQL, es posible especificar múltiples columnas de agrupación para organizar los resultados en grupos basados en el contenido de esas columnas. Esto permite obtener totales o resúmenes para cada combinación única de valores en esas columnas.

En SQL, es posible agrupar los resultados de consultas basándose en el contenido de dos o más columnas. Por ejemplo, *se puede calcular el total facturado por cada vendedor y a cada cliente el año pasado, ordenado primero por vendedor y luego por cliente*.

La sintaxis para lograr esto sería la siguiente:

```
SELECT V.COD_VENDEDOR, V.APE_VENDEDOR + ', ' + V.NOM_VENDEDOR 'VENDEDOR',
       C.APE_CLIENTE + ', ' + C.NOM_CLIENTE 'CLIENTE',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD)'TOTAL'
  FROM FACTURAS F, DETALLE_FACTURAS DF, VENDEDORES V, CLIENTES C
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
   AND V.COD_VENDEDOR = F.COD_VENDEDOR
   AND C.COD_CLIENTE = F.COD_CLIENTE
   AND YEAR(F.FECHA) = YEAR(GETDATE()) -1
 GROUP BY V.COD_VENDEDOR, V.APE_VENDEDOR + ', ' + V.NOM_VENDEDOR,
          C.APE_CLIENTE + ', ' + C.NOM_CLIENTE
 ORDER BY 2,3
```

	COD_VENDEDOR	VENDEDOR	CLIENTE	TOTAL
1	1	Carrizo, Martín	Abarca, Héctor	2894.00
2	1	Carrizo, Martín	Castillo, Marta Analía	1828.00
3	1	Carrizo, Martín	Luque, Elvira Josefa	12318.50
4	1	Carrizo, Martín	Morales, Santiago	30524.00
5	1	Carrizo, Martín	Paez, Roque	2160.00
6	1	Carrizo, Martín	Perez, Carlos Antonio	19366.00
7	1	Carrizo, Martín	Perez, Rodolfo	14380.00
8	1	Carrizo, Martín	Ruiz, Marcos	1628.50
9	2	Ledesma, Mariela	Abarca, Héctor	14305.50
10	2	Ledesma, Mariela	Castillo, Marta Analía	2120.00
11	2	Ledesma, Mariela	Luque, Elvira Josefa	56804.00

Imagen 4: Elaboración propia.

Esta consulta responde a la pregunta: "¿Cuánto vendió cada vendedor a cada cliente el año pasado?" En este ejemplo, es importante observar las columnas que se incluyen como agrupación en la cláusula **GROUP BY**, que son aquellas relacionadas con los vendedores y los clientes.

En la cláusula GROUP BY se incluyen las siguientes columnas:

- V.COD_VENDEDOR: Código del vendedor.
- V.APE_VENDEDOR + ', ' + V.NOM_VENDEDOR: Apellido y nombre del vendedor concatenados.
- C.APE_CLIENTE + ', ' + C.NOM_CLIENTE: Apellido y nombre del cliente concatenados.

Estas columnas se utilizan para agrupar los resultados y generar filas únicas para cada combinación de vendedor y cliente.

En la lista de selección de la consulta, se utilizan dos tipos de columnas:

1. **Columnas dentro de funciones de agregado**:
   - **SUM(DF.PRE_UNITARIO * DF.CANTIDAD)**: Esta columna calcula el total de ventas para cada combinación de vendedor y cliente. Se suma el resultado de multiplicar el precio unitario por la cantidad en la tabla DETALLE_FACTURAS.
2. **Columnas de agrupación:**
   - **V.COD_VENDEDOR**: Se incluye para mostrar el código del vendedor en el resultado.
   - **V.APE_VENDEDOR + ', ' + V.NOM_VENDEDOR**: Se incluye para mostrar el apellido y nombre del vendedor concatenados en el resultado.
   - **C.APE_CLIENTE + ', ' + C.NOM_CLIENTE**: Se incluye para mostrar el apellido y nombre del cliente concatenados en el resultado.

Es importante tener en cuenta que, en una consulta agrupada, todas las columnas en la lista de selección deben ser funciones de agregado o columnas de agrupación. Esto significa que las columnas seleccionadas deben estar involucradas en una función de agregado, como SUM, COUNT, AVG, entre otras, o deben formar parte de la cláusula GROUP BY.

Introducción a los procedimientos almacenados

En SQL Server, un procedimiento almacenado es un conjunto de instrucciones SQL que se almacenan en el servidor de bases de datos para su ejecución posterior. Estas instrucciones pueden ser consultas, comandos de manipulación de datos (como INSERT, UPDATE o DELETE) o incluso lógica de programación más compleja.

Los procedimientos almacenados se crean y almacenan en la base de datos, lo que permite su reutilización y ejecución eficiente. Proporcionan una forma conveniente de agrupar y organizar la lógica empresarial en el lado del servidor.

Un procedimiento almacenado puede aceptar parámetros de entrada y devolver resultados o valores de salida. Esto permite la personalización de la ejecución del procedimiento almacenado en función de los valores proporcionados en los parámetros.

Puede hacer referencia a objetos que no existen al momento de crearlo. Los objetos deben existir cuando se ejecute el procedimiento almacenado.

Los procedimientos almacenados en SQL Server pueden:

- Ser creados y almacenados en la base de datos para su ejecución posterior.
- ***Aceptar parámetros de entrada y devolver resultados o valores de salida.***
- Proporcionar seguridad al establecer permisos de acceso a nivel de procedimiento.

Un procedimiento almacenado en SQL Server puede contener una variedad de instrucciones SQL, pero existen algunas excepciones en cuanto a ciertos comandos de creación de objetos. Las siguientes instrucciones de creación: CREATE DEFAULT, CREATE PROCEDURE, CREATE RULE, CREATE TRIGGER y CREATE VIEW, ***no pueden incluirse directamente dentro de un procedimiento almacenado***, ya que están destinadas a crear objetos en sí mismas.

Sin embargo, un procedimiento almacenado puede interactuar con otros objetos ya creados, como tablas, vistas, funciones o trigger, utilizando las instrucciones adecuadas para manipular y consultar dichos objetos existentes. Esto significa que un procedimiento almacenado puede utilizar instrucciones como SELECT, INSERT, UPDATE o DELETE para interactuar con los datos de las tablas existentes.

Creación y ejecución de procedimientos almacenados

La creación y ejecución de procedimientos almacenados en SQL Server sin parámetros se realiza de la siguiente manera:

1. **Creación del procedimiento almacenado**: Para crear un procedimiento almacenado sin parámetros, se utiliza la sentencia **CREATE PROCEDURE**. Aquí se define el nombre del procedimiento almacenado y se escribe la lógica del procedimiento utilizando instrucciones SQL. Por ejemplo:

```
CREATE PROCEDURE NombreProcedimiento
AS
BEGIN
    -- Lógica del procedimiento
    -- Instrucciones SQL aquí
END;
```

2. **Compilación del procedimiento almacenado**: Después de crear el procedimiento almacenado, se debe compilar para validar la sintaxis y generar un plan de ejecución optimizado. Esto se hace automáticamente cuando se crea el procedimiento o cuando se modifica. Si hay errores de sintaxis, se mostrarán mensajes de error.
3. **Ejecución del procedimiento almacenado**: Una vez creado y compilado, el procedimiento almacenado se puede ejecutar utilizando la sentencia **EXECUTE** o su forma abreviada **EXEC**. Por ejemplo:

```
EXEC NombreProcedimiento;
```

NOTA: Por convención, a los procedimientos almacenados se les suele añadir un prefijo de "**sp_**" o "**pa_**" al momento de nombrarlos. Estos prefijos indican que se trata de un procedimiento almacenado en la base de datos.

Ejemplo de creación de procedimiento almacenado

A continuación, vemos un ejemplo, se muestra cómo crear un procedimiento almacenado llamado "pa_articulos_precios_mas100" *que mostrará todos los artículos de la librería con precios mayores a 100*:

```
CREATE PROCEDURE pa_articulos_precios_mas100
AS
BEGIN
    -- Lógica del procedimiento
    SELECT *
      FROM articulos
     WHERE pre_unitario > 100;
END;
```

En el ejemplo anterior, se utiliza la sentencia **CREATE PROCEDURE** para crear el procedimiento almacenado llamado "pa_articulos_precios_mas100". Dentro del procedimiento, se incluye la lógica que consiste en una consulta SELECT que recupera todos los artículos de la tabla "Artículos" donde el precio unitario sea mayor a 100.

Una vez creado el procedimiento almacenado, puedes ejecutarlo utilizando la sentencia **EXECUTE** o su forma abreviada **EXEC**:

```
EXEC pa_articulos_precios_mas100;
```

Al ejecutar el procedimiento, se mostrarán los resultados que cumplan con la condición establecida en la consulta SELECT, es decir, los artículos de la librería con precios mayores a 100, con todas sus columnas por no especificar los mismas.

	cod_articulo	descripcion	stock_minimo	stock	pre_unitario	observaciones
1	1	Lápiz Evolution HB2 * 4 u	NULL	150	160.00	NULL
2	2	Papel p/forrar Fantasía * 10 u	100	100	500.00	NULL
3	3	Papel p/forrar Araña * 10 u	20	155	600.00	Color rojo - azul - verde
4	4	Lápices Color cortos * 12 u	50	100	120.50	NULL
5	5	Fibras cortas * 6	50	50	125.30	NULL
6	6	Lápices Color largos * 12 u	NULL	0	130.00	NULL
7	8	Carpeta fibra negra - Tamaño rivadavia	NULL	54	202.90	3 anillos
8	9	Carpeta Fantasía - Tamaño Rivadavia	60	100	250.90	3 anillos
9	13	Repuesto Gloria rallado	150	120	641.00	400 hojas
10	14	Repuesto Gloria cuadriculado	20	90	641.00	400 hojas

Imagen 5: Elaboración propia.

Modificación de procedimientos almacenados

Para modificar un procedimiento almacenado existente en SQL Server, puedes seguir estos pasos:

1. **Utiliza la sentencia ALTER PROCEDURE**: Para realizar modificaciones en un procedimiento almacenado, se utiliza la sentencia **ALTER PROCEDURE** seguida del nombre del procedimiento que deseas modificar. Por ejemplo:

```
ALTER PROCEDURE NombreProcedimiento
AS
BEGIN
    -- Lógica actualizada del procedimiento
    -- Instrucciones SQL aquí
END;
```

2. **Realiza las modificaciones necesarias**: Dentro del bloque **BEGIN...END**, realiza las modificaciones necesarias en la lógica del procedimiento almacenado. Puedes agregar, eliminar o modificar las instrucciones SQL según tus requisitos.
3. **Compila el procedimiento almacenado**: Una vez realizadas las modificaciones, el procedimiento almacenado debe compilarse para validar la sintaxis y generar un nuevo plan de ejecución optimizado. La compilación se realiza automáticamente al ejecutar la sentencia **ALTER PROCEDURE**. Si hay errores de sintaxis, se mostrarán mensajes de error.
4. **Verifica las modificaciones**: Puedes revisar las modificaciones realizadas en el procedimiento almacenado para asegurarte de que se hayan implementado correctamente.

Es importante tener en cuenta que, al modificar un procedimiento almacenado, cualquier código o consulta que haga referencia a él también se verá afectado por los cambios realizados. Asegúrate de tener en cuenta las dependencias y de probar exhaustivamente el procedimiento modificado antes de implementarlo en un entorno de producción.

Ejemplo de modificación de procedimiento almacenado

Veamos un ejemplo, para modificar el procedimiento almacenado anterior y mostrar solo las columnas "descripcion", "pre_unitario", "observaciones" y "stock", puedes seguir estos pasos:

1. **Utiliza la sentencia ALTER PROCEDURE**: Utiliza la sentencia **ALTER PROCEDURE** seguida del nombre del procedimiento almacenado que deseas modificar. Por ejemplo:

```
ALTER PROCEDURE pa_articulos_precios_mas100
AS
BEGIN
    -- Lógica actualizada del procedimiento
    SELECT descripcion, pre_unitario, observaciones, stock
      FROM articulos
     WHERE pre_unitario > 100;
END;
```

2. **Realiza las modificaciones necesarias**: Dentro del bloque **BEGIN...END**, reemplaza la consulta SELECT existente por la nueva consulta que selecciona las columnas requeridas. En este caso, seleccionamos las columnas "descripcion", "pre_unitario", "observaciones" y "stock" de la tabla "Articulos".

3. **Compila el procedimiento almacenado**: La compilación se realiza automáticamente al ejecutar la sentencia **ALTER PROCEDURE**. Verifica que no haya errores de sintaxis.

Una vez modificado el procedimiento almacenado, puedes ejecutarlo de la misma manera que antes:

```
EXEC pa_articulos_precios_mas100;
```

Ahora, el procedimiento almacenado mostrará solo las columnas "descripcion", "pre_unitario", "observaciones" y "stock" de los artículos de la librería con precios mayores a 100.

	descripcion	pre_unitario	observaciones	stock
1	Lápiz Evolution HB2 * 4 u	160.00	NULL	150
2	Papel p/forrar Fantasía * 10 u	500.00	NULL	100
3	Papel p/forrar Araña * 10 u	600.00	Color rojo - azul - verde	155
4	Lápices Color cortos * 12 u	120.50	NULL	100
5	Fibras cortas * 6	125.30	NULL	50
6	Lápices Color largos * 12 u	130.00	NULL	0
7	Carpeta fibra negra - Tamaño rivadavia	202.90	3 anillos	54
8	Carpeta Fantasía - Tamaño Rivadavia	250.90	3 anillos	100
9	Repuesto Gloria rallado	641.00	400 hojas	120
10	Repuesto Gloria cuadriculado	641.00	400 hojas	90

Imagen 6: Elaboración propia.

Eliminación de procedimientos almacenados

Para eliminar un procedimiento almacenado en SQL Server, puedes seguir estos pasos:

1. **Utiliza la sentencia DROP PROCEDURE**: Para eliminar un procedimiento almacenado, se utiliza la sentencia **DROP PROCEDURE** seguida del nombre del procedimiento que deseas eliminar. Por ejemplo:

```
DROP PROCEDURE NombreProcedimiento;
```

2. **Ejecuta la sentencia DROP PROCEDURE**: Ejecuta la sentencia **DROP PROCEDURE** para eliminar el procedimiento almacenado de la base de datos. Asegúrate de tener los permisos adecuados para eliminar el procedimiento.

Una vez ejecutada la sentencia **DROP PROCEDURE**, el procedimiento almacenado se eliminará de la base de datos y no podrá ser utilizado posteriormente.

Es importante tener en cuenta que la eliminación de un procedimiento almacenado es una acción permanente y no se puede deshacer. Antes de eliminar un procedimiento almacenado, asegúrate de que ya no se necesite y de que no haya dependencias en otras partes del sistema que utilicen ese procedimiento.

Ejemplo de eliminación de procedimiento almacenado

Para eliminar el procedimiento almacenado modificado anteriormente, "pa_articulos_precios_mas100", puedes seguir estos pasos:

```
DROP PROCEDURE pa_articulos_precios_mas100;
```

Simplemente ejecuta la sentencia **DROP PROCEDURE** seguida del nombre del procedimiento almacenado que deseas eliminar: En este caso, ejecutar "DROP PROCEDURE pa_articulos_precios_mas100;" eliminará el procedimiento almacenado "pa_articulos_precios_mas100" de la base de datos.

Procedimientos almacenados con parámetros de entrada

Procedimientos almacenados con parámetros de entrada ***permiten la transferencia de información a través de variables en el procedimiento***. Para habilitar la funcionalidad de parámetros de entrada en un procedimiento almacenado, se deben declarar los parámetros como variables durante su creación.

La sintaxis utilizada es la siguiente:

```
CREATE PROCEDURE nombre_procedimiento
    @nombre_parametro tipo = valor_por_defecto
AS
    sentencias;
```

Los parámetros se definen después del nombre del procedimiento, precedidos por un signo de arroba **(@)**. Los parámetros son variables locales al procedimiento, lo que significa que solo existen dentro del mismo. Pueden declararse varios parámetros separados por comas.

Cuando se ejecuta un procedimiento, se deben proporcionar valores para cada uno de los parámetros en el orden en que fueron definidos, a menos que se haya establecido un valor por defecto. En caso de haber un valor por defecto, los parámetros pueden omitirse al llamar al procedimiento.

Después de definir un parámetro y su tipo, se puede especificar un valor por defecto. Este valor será utilizado por el procedimiento cuando se ejecute sin proporcionar valores para los parámetros. El valor por defecto puede ser **"NULL"** o una constante. También es posible utilizar comodines si el procedimiento utiliza la cláusula **"LIKE"** en su lógica.

Ejemplo para Crear un Procedimiento Almacenado con Parámetros (E)

Veamos un ejemplo, Se solicita crea un nuevo procedimiento almacenado en la base de datos 'Libreria' que recibe dos precios unitarios como parámetros de entrada. El procedimiento tiene como objetivo mostrar los artículos cuyos precios estén dentro del rango especificado.

```
-- Ejemplo de creación de un procedimiento que recibe parámetros.
CREATE PROCEDURE pa_articulos_precios
  @precio1 money,
  @precio2 money
AS
BEGIN
  SELECT descripcion, pre_unitario, observaciones
    FROM articulos
   WHERE pre_unitario BETWEEN @precio1 AND @precio2;
END;

-- Ejecución del procedimiento especificando los valores para los
parámetros
-- Opción 1: Pasando valores por posición
EXEC pa_articulos_precios 100, 200;

-- Opción 2: Pasando valores por nombre de parámetro
EXEC pa_articulos_precios @precio1 = 100, @precio2 = 200;

-- Opción 3: Pasando valores por nombre de parámetro en orden alterado
EXEC pa_articulos_precios @precio2 = 200, @precio1 = 100;
```

En este ejemplo, se crea el procedimiento almacenado "pa_articulos_precios" que recibe dos parámetros de tipo money: "**@precio1**" y "**@precio2**". El procedimiento realiza una consulta en la tabla "articulos" para seleccionar los artículos cuyos precios estén dentro del rango especificado.

Luego, se muestra cómo ejecutar el procedimiento de dos maneras diferentes:

1. Pasando valores por posición: utilizando la sentencia "EXEC pa_articulos_precios" seguida de los valores para los parámetros separados por comas.
2. Pasando valores por nombre de parámetro: utilizando la sentencia "EXEC pa_articulos_precios" seguida de los valores para los parámetros especificando el nombre del parámetro seguido del símbolo **"="** y el valor correspondiente.

En ambos casos, se pueden pasar los valores en cualquier orden siempre que se especifique el nombre del parámetro correctamente.

Recuerda que este es solo un ejemplo y puedes adaptar la lógica y la estructura del procedimiento según tus necesidades y el diseño de tu base de datos.

Ejemplo de Modificación de Procedimiento Almacenado con Parámetros (E)

Mejor el procedimiento, ahora modificar el procedimiento que permita omitir los valores para los parámetros se debe, al crear el procedimiento, definir valores por defecto para cada parámetro entre 100 y 200:

```
ALTER PROCEDURE pa_articulos_precios
  @precio1 money = 100,
  @precio2 money = 200
AS
BEGIN
  -- Lógica del procedimiento utilizando los parámetros
  SELECT descripcion, pre_unitario, observaciones
  FROM articulos
  WHERE pre_unitario BETWEEN @precio1 AND @precio2;
END;
```

En este caso, se están asignando valores por defecto de 100 y 200 a los parámetros **@precio1** y **@precio2**, respectivamente. Si no se proporcionan valores al llamar al procedimiento, estos valores por defecto serán utilizados.

A continuación, te muestro cómo ejecutar el procedimiento con las nuevas modificaciones:

```
-- Opción 1: No se proporcionan valores, se utilizan los valores por
defecto
EXEC pa_articulos_precios;

-- Opción 2: Se proporciona solo un valor
EXEC pa_articulos_precios 150;

-- Opción 3: Se proporcionan ambos valores
EXEC pa_articulos_precios 50, 250;
```

En la opción 1, no se proporcionan valores y se utilizarán los valores por defecto de 100 y 200.

En la opción 2, se proporciona solo un valor (150) y **se asume que se refiere al primer parámetro** (@precio1). El segundo parámetro (@precio2) tomará el valor por defecto de 200.

En la opción 3, se proporcionan ambos valores (50 y 250) y se asignan al primer y segundo parámetro respectivamente.

Recuerda que, si deseas llamar al procedimiento utilizando los nombres de los parámetros, puedes hacerlo de la siguiente manera:

```
EXEC pa_articulos_precios @precio1 = 150;
EXEC pa_articulos_precios @precio2 = 250, @precio1 = 50;
```

IMPORATANTE: Una pregunta que uno podemos hacer es la siguiente:

🤔¿Cuál es el resultado si se envía un primer parámetro con un valor mayor al segundo parámetro?

😃La respuesta seria; si se envía un valor mayor para el primer parámetro (@precio1) y un valor menor para el segundo parámetro (@precio2) al ejecutar el procedimiento pa_articulos_precios, la consulta en la cláusula WHERE **no retornará ningún resultado**, ya que no habrá ningún valor en la columna pre_unitario que esté entre esos dos valores invertidos.

Por ejemplo, si ejecutas el procedimiento de la siguiente manera:

```
EXEC pa_articulos_precios @precio1 = 200, @precio2 = 100;
```

La condición WHERE pre_unitario BETWEEN @precio1 AND @precio2 buscará registros donde pre_unitario esté entre 200 y 100, pero ningún valor puede cumplir esa condición. Por lo tanto, la consulta no retornará ningún resultado.

Es importante asegurarse de enviar los parámetros en el orden correcto o ajustar la lógica del procedimiento para manejar el escenario en el que el primer parámetro sea mayor que el segundo.

Podrías intercambiar los valores si detectas que el primer parámetro es mayor que el segundo dentro del cuerpo del procedimiento, o podrías generar un mensaje de error o manejarlo de alguna otra forma según tus necesidades específicas.

Lo cual es un tema para investigar solos o esperar a ver en LCII manejo de errores. ☺

Ejemplo de procedimiento almacenado (E) para usar la lógica de un patrón de búsqueda

Continuaremos explorando los procedimientos almacenados que hacen uso de parámetros de entrada. Anteriormente, presentamos un ejemplo que incluía parámetros con valores por defecto. Ahora, nos adentraremos en el uso del patrón de búsqueda **LIKE** junto con parámetros de entrada. Para ilustrar esta situación, crearemos un nuevo procedimiento almacenado en nuestra base de datos "librería". El propósito de este procedimiento es buscar todos los artículos que coincidan con la descripción proporcionada como parámetro.

A continuación, se muestra el código del procedimiento almacenado:

```
CREATE PROCEDURE BuscarArticulosPorDescripcion
    @descripcion VARCHAR(100) = '%'
AS
BEGIN
    SELECT descripcion, pre_unitario, observaciones
      FROM articulos
     WHERE descripcion LIKE '%' + @descripcion + '%';
END;
```

Aquí tienes una explicación de su funcionamiento:

- El parámetro de entrada **@descripcion** se define como **VARCHAR(100)** y tiene un valor predeterminado de '**%**'. Esto significa que, si no se proporciona ningún valor para el parámetro al llamar al procedimiento, se asumirá '**%**' como valor predeterminado, lo que resultará en una búsqueda de todas las descripciones.
- La cláusula **WHERE descripcion LIKE '%' + @descripcion + '%'** se utiliza para filtrar los registros en función de la descripción proporcionada. Aquí es donde se aplica el patrón de búsqueda LIKE con los comodines '**%**'. El símbolo '**%**' se utiliza para representar cualquier número de caracteres (incluyendo ninguno), lo que permite buscar descripciones que contengan la cadena proporcionada en cualquier posición.

En resumen, el procedimiento almacenado "BuscarArticulosPorDescripcion" busca artículos en la tabla "articulos" basándose en una descripción proporcionada como parámetro. Si no se proporciona ninguna descripción específica, se seleccionarán todos los artículos. La cláusula **LIKE** con los comodines '**%**' permite buscar descripciones que contengan la cadena proporcionada en cualquier posición.

Aquí tienes dos ejemplos de ejecución del procedimiento almacenado "BuscarArticulosPorDescripcion" con valores y sin valores proporcionados para el parámetro **@descripcion**:

1. Ejecución **con un valor específico** para **@descripcion**:

```
EXEC BuscarArticulosPorDescripcion 'car';
```

En este caso, se busca todos los artículos cuyas descripciones contengan la palabra "**car**". La cláusula **LIKE** en la consulta del procedimiento almacenado buscará coincidencias que tengan la cadena "**car**" en cualquier posición de la descripción.

	descripcion	pre_unitario	observaciones
1	Carpeta fibra negra - Tamaño rivadavia	202.90	3 anillos
2	Carpeta Fantasía - Tamaño Rivadavia	250.90	3 anillos
3	Cartuchera Lata	565.50	2 pisos
4	Cartuchera de Tela	262.60	NULL

Imagen 7: Elaboración propia.

2. Ejecución **sin proporcionar un valor** para @descripcion:

```
EXEC BuscarArticulosPorDescripcion;
```

Cuando no se proporciona un valor específico para **@descripcion**, se utilizará el valor predeterminado '**%**'. Esto significa que el procedimiento almacenado buscará todos los artículos, ya que la cláusula **WHERE** en la consulta **WHERE descripcion LIKE '%' + @descripcion + '%'** se convertirá en **WHERE descripcion LIKE '%%'**, lo cual coincide con cualquier descripción.

	descripcion	pre_unitario	observaciones
1	Lápiz Evolution HB2 * 4 u	160.00	NULL
2	Papel p/forrar Fantasía * 10 u	500.00	NULL
3	Papel p/forrar Araña * 10 u	600.00	Color rojo - azul - verde
4	Lápices Color cortos * 12 u	120.50	NULL
5	Fibras cortas * 6	125.30	NULL
6	Lápices Color largos * 12 u	130.00	NULL
7	Separadores tamaño rivadavia * 6 u	85.70	Motivos de Disney
8	Carpeta fibra negra - Tamaño rivadavia	202.90	3 anillos
9	Carpeta Fantasía - Tamaño Rivadavia	250.90	3 anillos
10	Adhesivo sintético 30 gr	56.00	NULL

Imagen 8: Elaboración propia

Ejemplo de procedimiento almacenado (E) que utiliza una estructura de control alternativa simple.

En un procedimiento almacenado, se puede emplear una estructura de control alternativa simple utilizando la sentencia **IF/ELSE** para validar una condición y ejecutar una serie de acciones únicamente si la condición es verdadera. También es posible realizar otras acciones si la condición es falsa.

En SQL Server, puedes utilizar la estructura de control IF/ELSE para ejecutar diferentes bloques de código en función de una condición. Aquí tienes un ejemplo de cómo usar IF/ELSE:

```
IF condición
   BEGIN
      -- Acciones a realizar si la condición es verdadera
   END
ELSE
   BEGIN
      -- Acciones a realizar si la condición es falsa
   END
```

Por ejemplo, supongamos que deseamos generar un listado de artículos cuyo precio sea mayor a un valor determinado si se ingresa la opción 1. En caso de ingresar otra opción, el listado mostrará los artículos cuyo precio sea menor a dicho valor.

```
CREATE PROCEDURE pa_generar_listado_precios
    @opcion INT = 1,
    @precio MONEY = 100
AS
BEGIN
    IF @opcion = 1
        BEGIN
            -- Acciones si la opción es 1 (mayores a un precio)
            SELECT descripcion, pre_unitario, observaciones
              FROM articulos
              WHERE pre_unitario > @precio;
        END
    ELSE
        BEGIN
            -- Acciones si la opción no es 1 (menores a un precio)
            SELECT descripcion, pre_unitario, observaciones
              FROM articulos
              WHERE pre_unitario < @precio;
        END
END;
```

En este ejemplo, se crea el procedimiento almacenado llamado "**pa_generar_listado_precios**" con dos parámetros de entrada modificados: **@opcion** y **@precio**.

El parámetro **@opcion** se define como **INT** y tiene un valor predeterminado de 1. Esto significa que, si no se proporciona ningún valor para el parámetro al llamar al procedimiento, se asumirá 1 como valor predeterminado.

El parámetro **@precio** se define como **MONEY** y tiene un valor predeterminado de 100. Si no se proporciona un valor específico para el parámetro, se utilizará 100 como valor predeterminado.

Dentro del procedimiento, se utiliza la estructura de control alternativa simple y doble con la sentencia **IF/ELSE**. Si la condición **@opcion = 1** se evalúa como verdadera, se ejecutarán las acciones definidas en el bloque BEGIN y END correspondiente a la opción 1, que en este caso es seleccionar los artículos cuyo precio es mayor al valor proporcionado.

Si la condición **@opcion = 1** se evalúa como falsa, se ejecutarán las acciones definidas en el bloque BEGIN y END correspondiente al bloque **ELSE**, que en este caso es seleccionar los artículos cuyo precio es menor al valor proporcionado.

Ejecución del procedimiento almacenado.

Puedes ejecutar este procedimiento almacenado **sin proporcionar valores para los parámetros**, lo cual utilizará los valores predeterminados establecidos. Por ejemplo:

```
EXEC pa_generar_listado_precios;
```

Este código generará un listado de artículos cuyo precio sea mayor a 100, ya que se seleccionará el bloque de acciones correspondiente a la opción 1 y se utilizará el precio predeterminado de 100.

	descripcion	pre_unitario	observaciones
1	Lápiz Evolution HB2 * 4 u	160.00	NULL
2	Papel p/forrar Fantasía * 10 u	500.00	NULL
3	Papel p/forrar Araña * 10 u	600.00	Color rojo - azul - verde
4	Lápices Color cortos * 12 u	120.50	NULL
5	Fibras cortas * 6	125.30	NULL
6	Lápices Color largos * 12 u	130.00	NULL
7	Carpeta fibra negra - Tamaño rivadavia	202.90	3 anillos
8	Carpeta Fantasía - Tamaño Rivadavia	250.90	3 anillos
9	Repuesto Gloria rallado	641.00	400 hojas
10	Repuesto Gloria cuadriculado	641.00	400 hojas

Imagen 9: Elaboración propia

Otra opción es un ejemplo de ejecución del procedimiento almacenado "pa_generar_listado_precios" **con un valor específico para el parámetro** **@precio**:

```
EXEC pa_generar_listado_precios @precio = 500.00;
```

En este caso, se ejecuta el procedimiento almacenado "pa_generar_listado_precios" con el valor 500.00 para el parámetro **@precio**. No se proporciona un valor para el parámetro **@opcion**, por lo que se utilizará su valor predeterminado, que es 1.

Dado que el valor de **@opcion** es igual a 1, se ejecutarán las acciones definidas en el bloque **IF** del procedimiento. En este caso, se seleccionarán los artículos cuyo precio sea mayor a 500.00, ya que se utilizará el bloque SELECT correspondiente al bloque **IF**.

	descripcion	pre_unitario	observaciones
1	Papel p/forrar Araña * 10 u	600.00	Color rojo - azul - verde
2	Repuesto Gloria rallado	641.00	400 hojas
3	Repuesto Gloria cuadriculado	641.00	400 hojas
4	Cartuchera Lata	565.50	2 pisos
5	Resma hoja A4	585.00	NULL

Imagen 10: Elaboración propia

Aquí tienes un ejemplo de ejecución del procedimiento almacenado "pa_generar_listado_precios" **con valores específicos proporcionados para los parámetros**:

```
EXEC pa_generar_listado_precios @opcion = 2, @precio = 75.00;
```

En este caso, se ejecuta el procedimiento almacenado "pa_generar_listado_precios" con el valor 2 para el parámetro **@opcion** y 75.00 para el parámetro **@precio**.

Debido a que el valor de **@opcion** no es igual a 1, se ejecutarán las acciones definidas en el bloque **ELSE** del procedimiento. En este caso, se seleccionarán los artículos cuyo precio sea menor a 75.00, ya que se utilizará el bloque **SELECT** correspondiente al bloque **ELSE**.

En este ejemplo, se generará un listado de artículos cuyo precio sea menor a 75.00 debido a los valores proporcionados para los parámetros.

	descripcion	pre_unitario	observaciones
1	Adhesivo sintético 30 gr	56.00	NULL
2	Cuaderno Tamaño rivadavia rayado	52.70	NULL
3	Cuaderno Tamaño rivadavia cuadriculado	52.70	NULL
4	Lápices con goma * 2 u	65.50	NULL
5	Lapicera Bic Azul trazo fino	54.99	NULL

Imagen 11: Elaboración propia

Recuerda que puedes ajustar los valores de los parámetros según tus necesidades de búsqueda.

BIBLIOGRAFÍA

- Gorman K., Hirt A., Noderer D., Rowland-Jones J., Sirpal A., Ryan D. & Woody B (2019) Introducing Microsoft SQL Server 2019. Reliability, scalability, and security both on premises and in the cloud. Packt Publishing Ltd. Birmingham UK
- Microsoft. SQL Server 2016. Disponible en: https://www.microsoft.com/es-es/sql-server/sql-server-2016
- Opel, A. & Sheldon, R. (2010). Fundamentos de SQL. Madrid. Editorial Mc Graw Hill
- Varga S., Cherry D., D'Antoni J. (2016). Introducing Microsoft SQL Server 2016 Mission-Critical Applications, Deeper Insights, Hyperscale Cloud. Washington. Microsoft Press

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera: Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.